

IWT

(26CSE0106)

Dr. Kamal

**Chitkara University Institute of Engineering
and Technology
Chitkara University**

After this lecture, students will be able to:

- Understand Web Technologies
- Explain WWW evolution
- Describe Internet Architecture
- Differentiate Frontend & Backend
- Understand HTTP/HTTPS
- Explain DNS, URL, Domain Names
- Understand Web Hosting

What is Web Technology?

Web Technology refers to the collection of technologies, tools, programming languages, and protocols used to create, develop, and maintain websites and web applications that run over the Internet.

Importance

- Enables worldwide access to information and services.
- Supports online communication, learning, shopping, and banking.
- Provides interactive and dynamic web applications.
- Forms the foundation of modern digital businesses.

Characteristics

- Platform Independent: Works on Windows, macOS, Linux, Android, and iOS.
- Client-Server Architecture: Browser (client) communicates with the web server.
- Interactive: Supports multimedia, animations, and user interaction.
- Scalable: Can handle a small website or millions of users.
- Accessible: Available anytime and anywhere with an Internet connection.

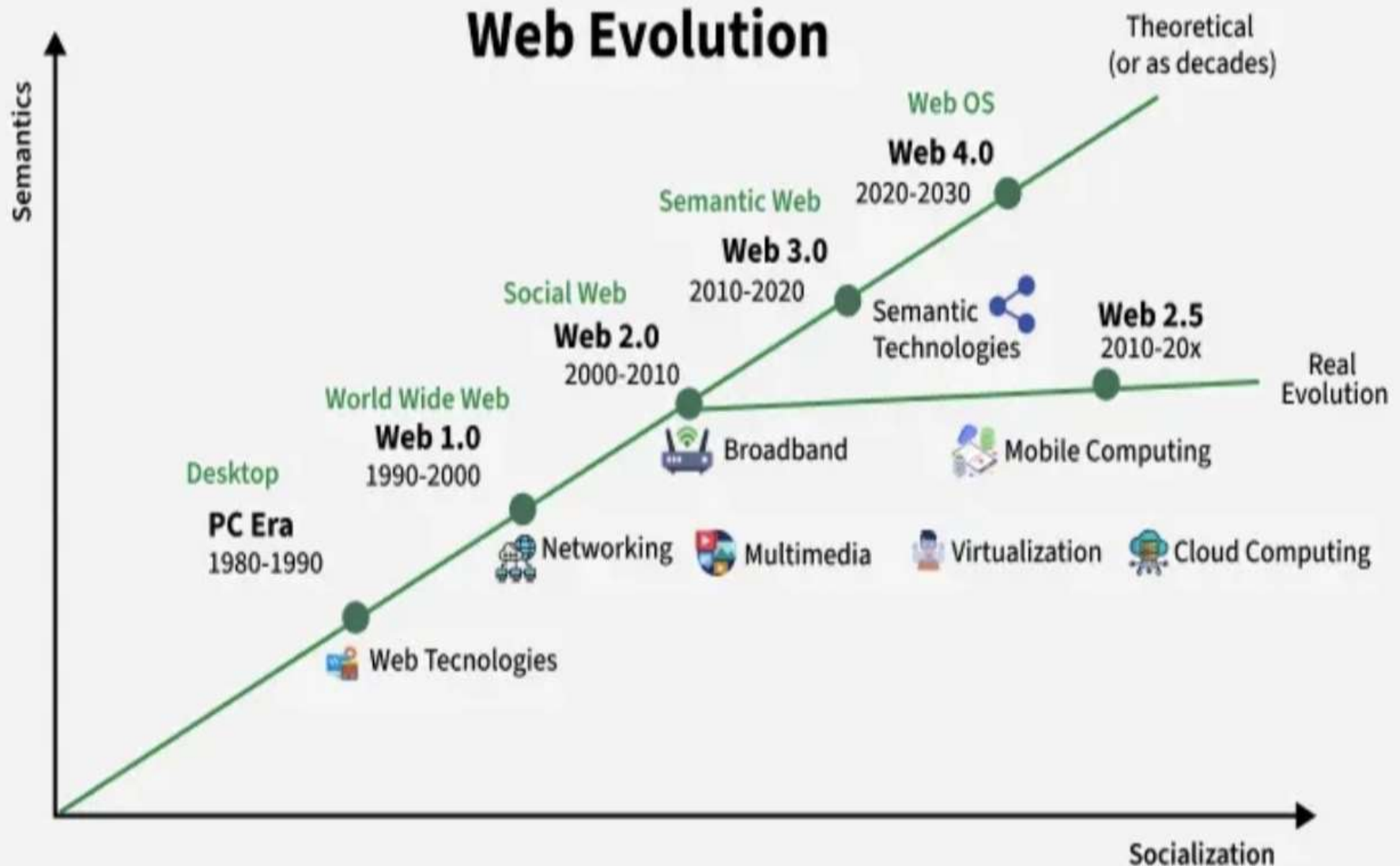
Applications

- E-commerce websites (Amazon, Flipkart)
- Social networking sites (Facebook, Instagram)
- Online banking and digital payments
- E-learning platforms (Coursera, NPTEL)
- Video streaming services (YouTube, Netflix)
- Cloud-based applications (Google Docs, Microsoft 365)

Evolution of the World Wide Web

Year	Development	Explanation
1969	ARPANET	ARPANET (Advanced Research Projects Agency Network) was the first computer network developed by the U.S. Department of Defense. It became the foundation of the Internet.
1983	TCP/IP	TCP/IP (Transmission Control Protocol/Internet Protocol) became the standard communication protocol, allowing different computer networks to communicate with each other. This marked the beginning of the modern Internet.
1989	WWW Proposed by Tim Berners-Lee	Tim Berners-Lee proposed the World Wide Web (WWW) at CERN. He introduced the concepts of HTML, HTTP, and URLs, making it easier to access and share information over the Internet.
1991	First Website	The world's first website was launched by Tim Berners-Lee. It explained what the World Wide Web was and how people could create web pages.
1995	Dynamic Websites	Websites became interactive using technologies like PHP, ASP, JavaScript, and databases. Users could log in, search, and interact with websites instead of viewing only static pages.
2005	Web 2.0	Web 2.0 introduced user-generated content and social interaction. Popular examples include Facebook, YouTube, Wikipedia, and blogs, where users could create and share content.
2010	HTML5	HTML5 was introduced with support for audio, video, graphics (Canvas), local storage, and responsive web design, reducing the need for plugins like Flash.
2020	Progressive Web Apps (PWAs)	PWAs combine the features of websites and mobile apps. They can work offline, load quickly, send push notifications, and be installed on devices without using an app store.

Cont..



What are Web Technologies?

- Web technologies are the various tools and techniques that are utilised in the process of communication between different types of devices over the internet.
- Let's break it down into two pieces: 'web' and 'technology'.
- The web, in this case, refers to the World Wide Web, more commonly known as WWW. It first came into being in 1989 when famous scientist and engineer, Tim Berners-Lee, came up with an efficient mechanism to share resources between scientists all over the world.

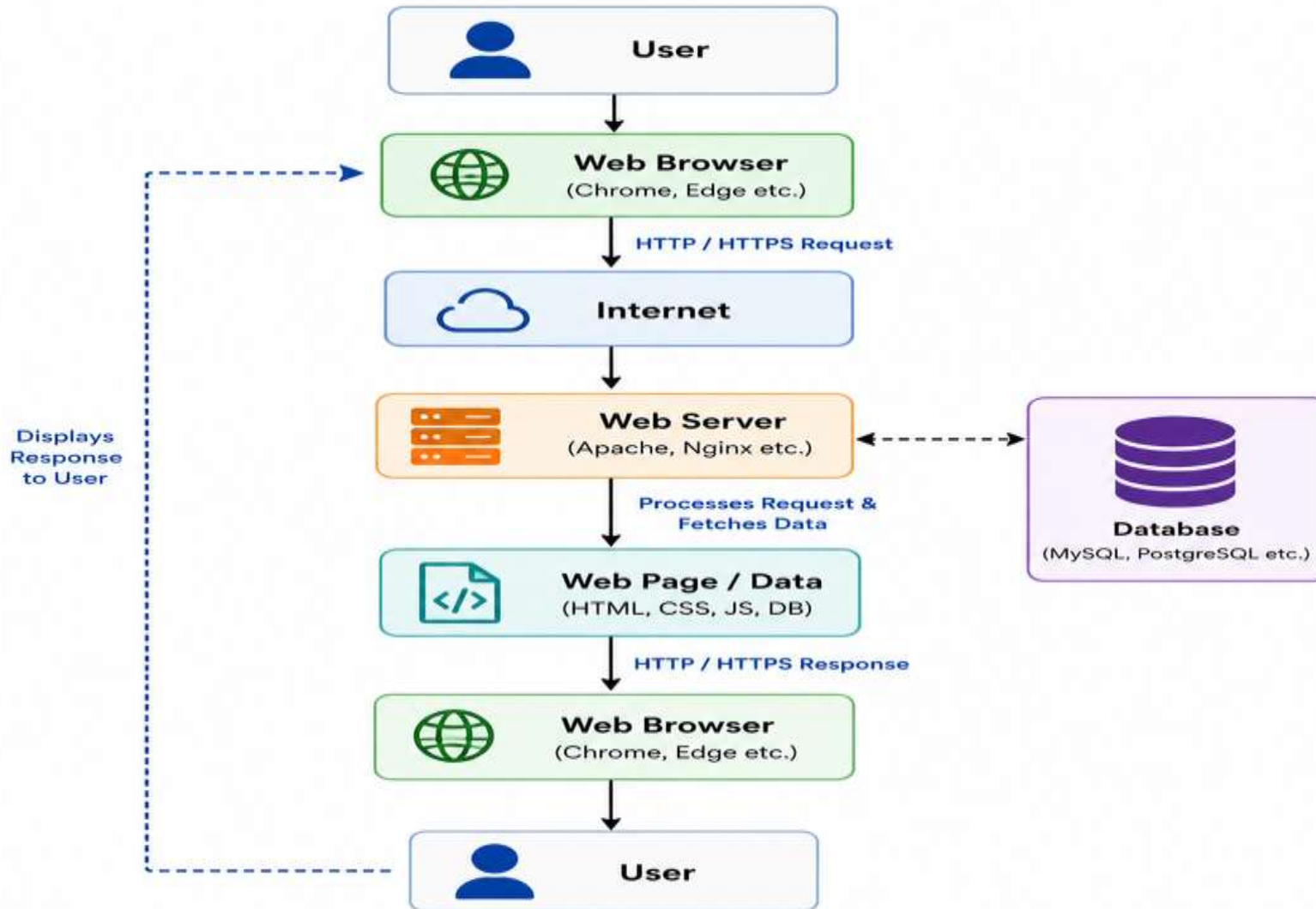
What is Internet Architecture?

- Internet architecture refers to the overall design and structure of the Internet.
- The term Internet comes from Inter-Networking, which means connecting multiple independent networks.
- It is based on the TCP/IP (Transmission Control Protocol/Internet Protocol) protocol suite.
- TCP/IP provides a standard set of communication rules that enable different types of networks to communicate with each other.
- This architecture allows networks with different hardware, software, and technical designs to exchange data seamlessly.
- It ensures reliable, scalable, and universal communication across the Internet.

Cont..



WEB WORKING ARCHITECTURE



You: “Show my result.”



Browser: Sends request



Internet: Carries request



Web Server: Receives request



Database: Finds your marks



Web Server: Creates result page



Browser: Receives the page



You: See your result

Application Protocols

- Provides network services directly to end-user applications.

Transport Protocols (TCP/UDP)

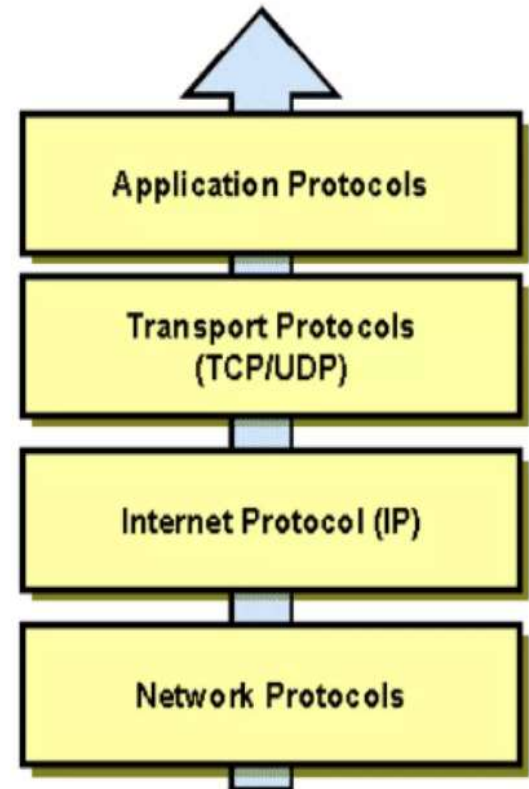
- Ensures reliable or fast end-to-end data communication between devices.

Internet Protocol (IP)

- Handles logical addressing and routes data packets across networks.

Network Protocols

- Manages the physical transmission of data over network media.



- 1. Application Protocols —
 - This is the layer closest to the user.
 - It provides services that applications use to communicate over the Internet.
- Examples:
- HTTP/HTTPS → Web browsing
 - FTP → File transfer
 - SMTP → Sending emails
 - DNS → Finding the IP address of a website
- **Simple meaning:** What service does the user want?

- 2. Transport Protocols
 - The main protocols are:
 - TCP → Reliable delivery
 - UDP → Faster delivery but less reliable
- TCP makes sure data reaches the destination correctly and in the proper order.

Simple meaning: How should the data be delivered?

- 3. Internet Protocol (IP)
 - The IP layer is responsible for addressing and routing.
 - Every device on a network has an IP address.
 - For example:
 - 192.168.1.10
 - IP helps decide where the data should go.

Simple meaning: Where should the data go?

- 4. Network Protocols
 - This is the lowest layer shown in the diagram.
 - It deals with the actual movement of data through the network, such as Ethernet or Wi-Fi.

Simple meaning: How does the data physically/network-wise travel?

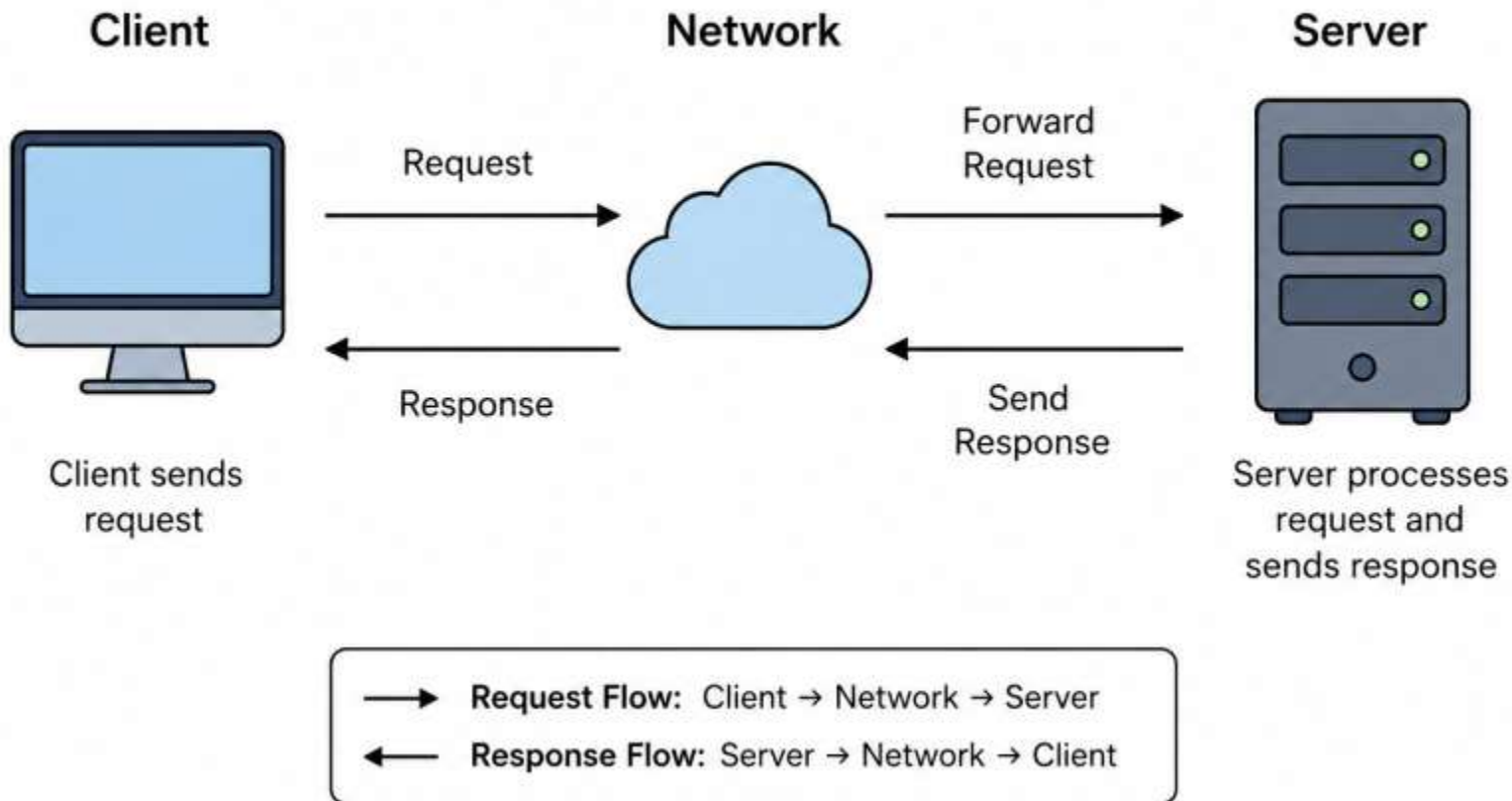
Internet vs World Wide Web (WWW)



Internet	World Wide Web (WWW)
A network of networks connecting millions of computers worldwide.	A collection of websites and web pages available on the Internet.
It is the hardware and communication infrastructure.	It is an information system that runs on the Internet.
Uses TCP/IP (Transmission Control Protocol/Internet Protocol) for communication.	Uses HTTP/HTTPS (Hypertext Transfer Protocol/Secure) to access web pages.
Supports many services like Email, FTP, VoIP, and the Web.	Provides access to websites, online documents, images, videos, and web applications.
Invented in the 1960s–1980s as a global communication network.	Invented by Tim Berners-Lee in 1989 to share information through web pages.



Simple Client-Server Architecture



What is a Client?

A client is a computer, laptop, workstation, or mobile device that users use to access applications and services. It sends requests to a server and relies on the server for resources such as files, databases, printers, and applications.

What is a Server?

A Server is a computer or software program that provides services, resources, or data to other computers (called clients) over a network.

Example: Opening Google

- When you open a website:
- Application: HTTP/HTTPS → “I want this webpage.”
↓
- Transport: TCP → “Deliver the data reliably.”
↓
- Internet: IP → “Send it to the correct computer.”
↓
- Network: Wi-Fi/Ethernet → “Move the data through the network.”

In short:

- Application → Service
- Transport → Delivery
- IP → Address/Route
- Network → Actual transmission

2-Tier Architecture (Client ↔ Server)

- In a 2-tier architecture, the application is divided between two systems:
 - Client (User's Computer)
 - Server (Database Server)

3-tier architecture

- **3-tier architecture**, the application is divided into three separate layers, each running independently.

Presentation Layer

Application (Business Logic) Layer

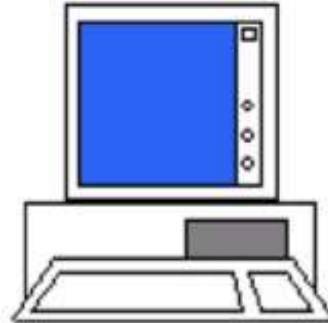
Database Layer

2-Tier Architecture



First Tier:

Client

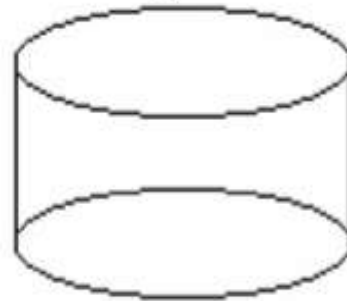


Tasks/Services

- User Interface
- Presentation services
- *Application services*

Second Tier:

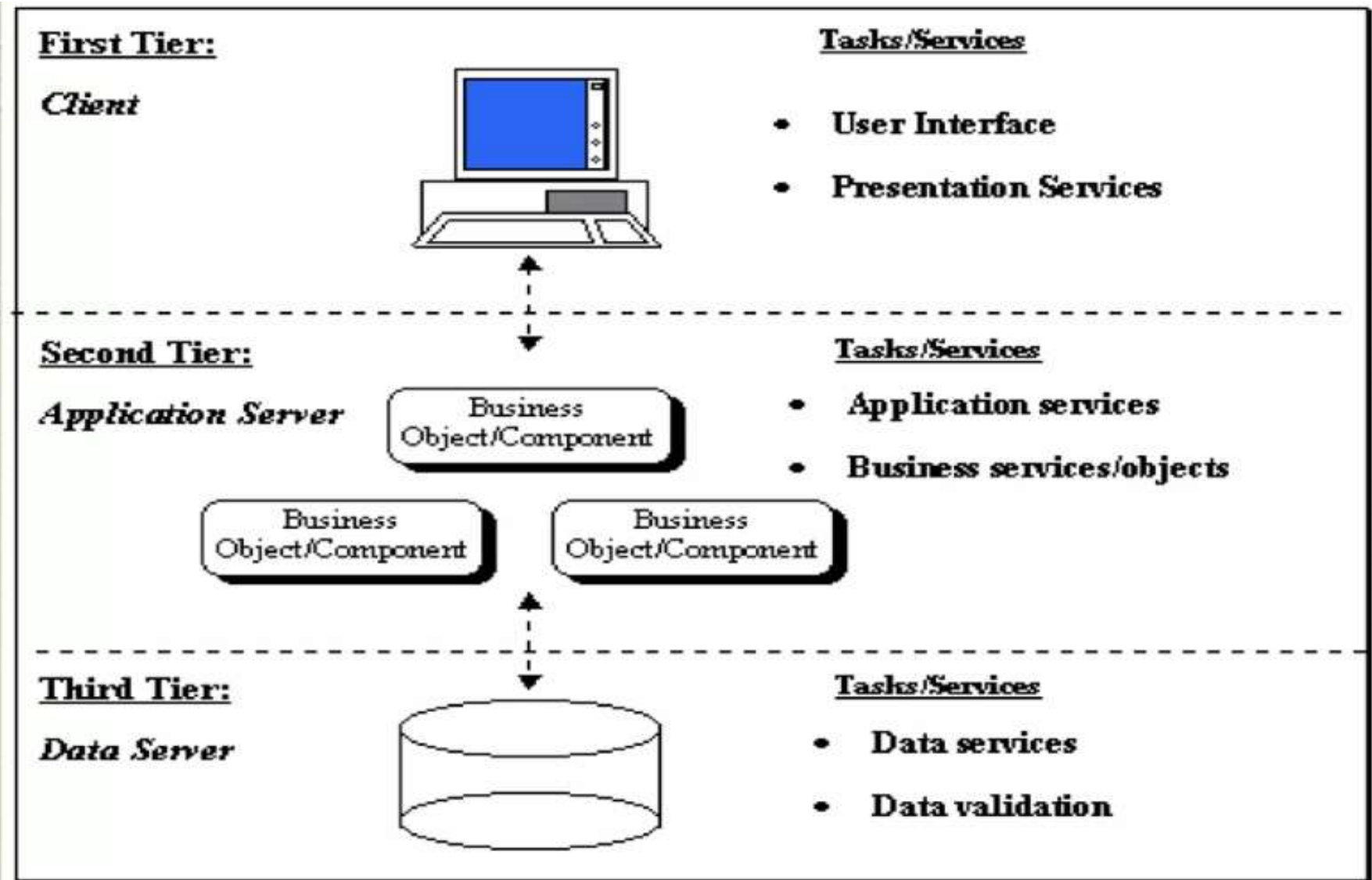
Data Server



Tasks/Services

- *Application services*
- *Business services*
- Data services

3-tier architecture



Difference Between 2-Tier and 3-Tier Architecture

Feature	2-Tier	3-Tier
Number of Layers	2	3
Business Logic	Client	Application Server
Database	Server	Separate Database Server
Security	Lower	Higher
Scalability	Limited	High
Performance	Suitable for small applications	Suitable for large enterprise applications
Maintenance	Difficult	Easier
Examples	MS Access, Desktop Applications	Amazon, Flipkart, Banking Systems

- Frontend is the piece of the website users can see and interact with, for example, the graphical UI (GUI) and the command line, including the design, navigating menus, texts, images, videos, etc.
- Backend, in actuality, is the piece of the site clients can't see and interface with. It's about how everything works.

Frontend vs. Backend

- **Frontend:** The visible side. It's what users interact with – from buttons to animations.
- **Backend:** The invisible side. Handles data, logic, and ensures everything works properly behind the scenes.
- **Collaboration:** Both need to work in perfect harmony for smooth user experiences!

Popular Frontend and Backend Technologies



HTML: The structure - It's like the skeleton of the website.



CSS: The styling - It dresses up the skeleton, making it visually appealing.



JavaScript: The magic - Brings interactivity (buttons, forms, animations).



React: Efficient, reusable components - Powerhouse for dynamic user interfaces.



Node.js: Run JavaScript on the server side - a favorite for real-time apps!



Express.js: Minimal, flexible framework for building APIs with Node.js.



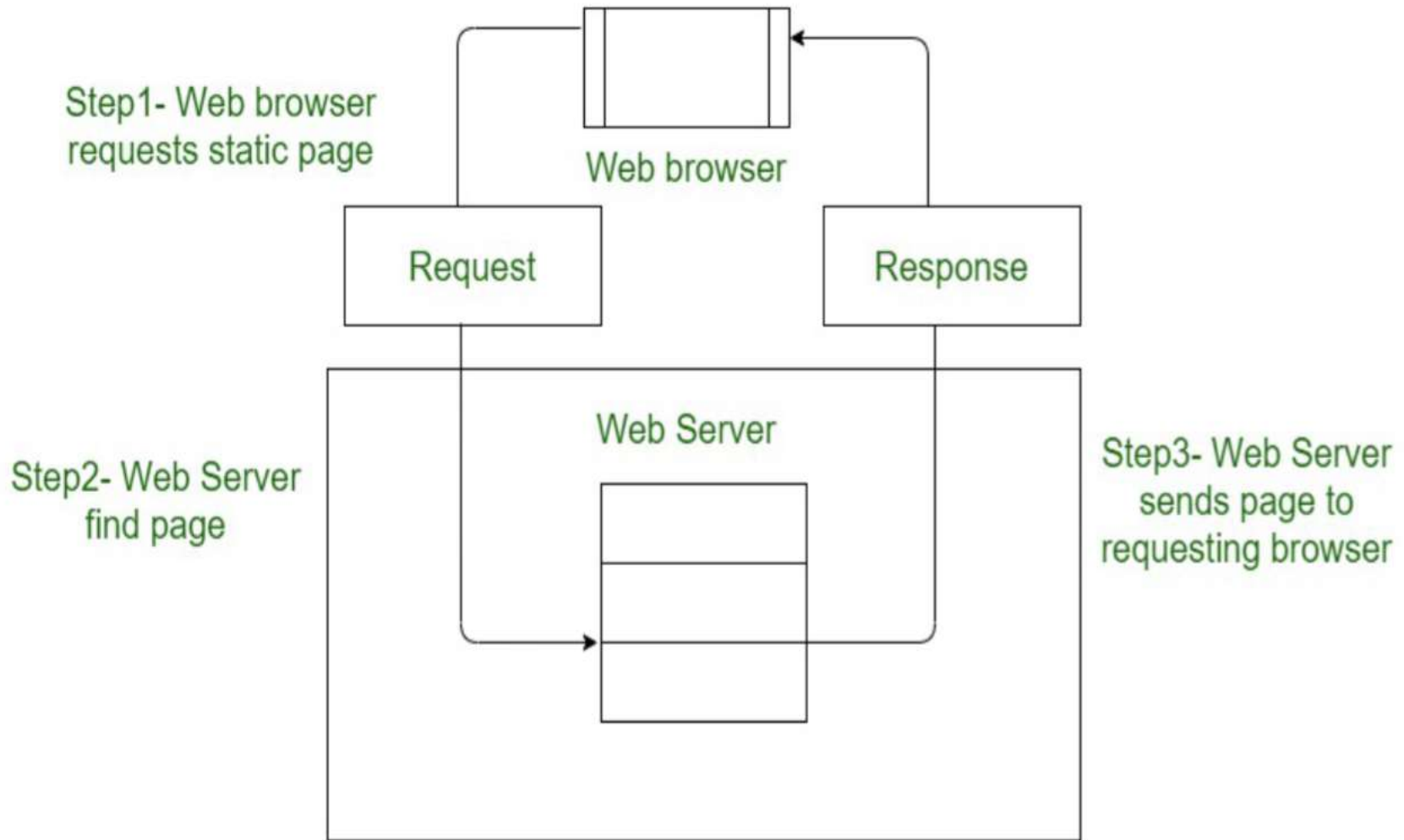
Python (Django/Flask): Simple syntax for rapid backend development and flexible web frameworks.



PHP: The tried-and-true scripting language for dynamic websites.

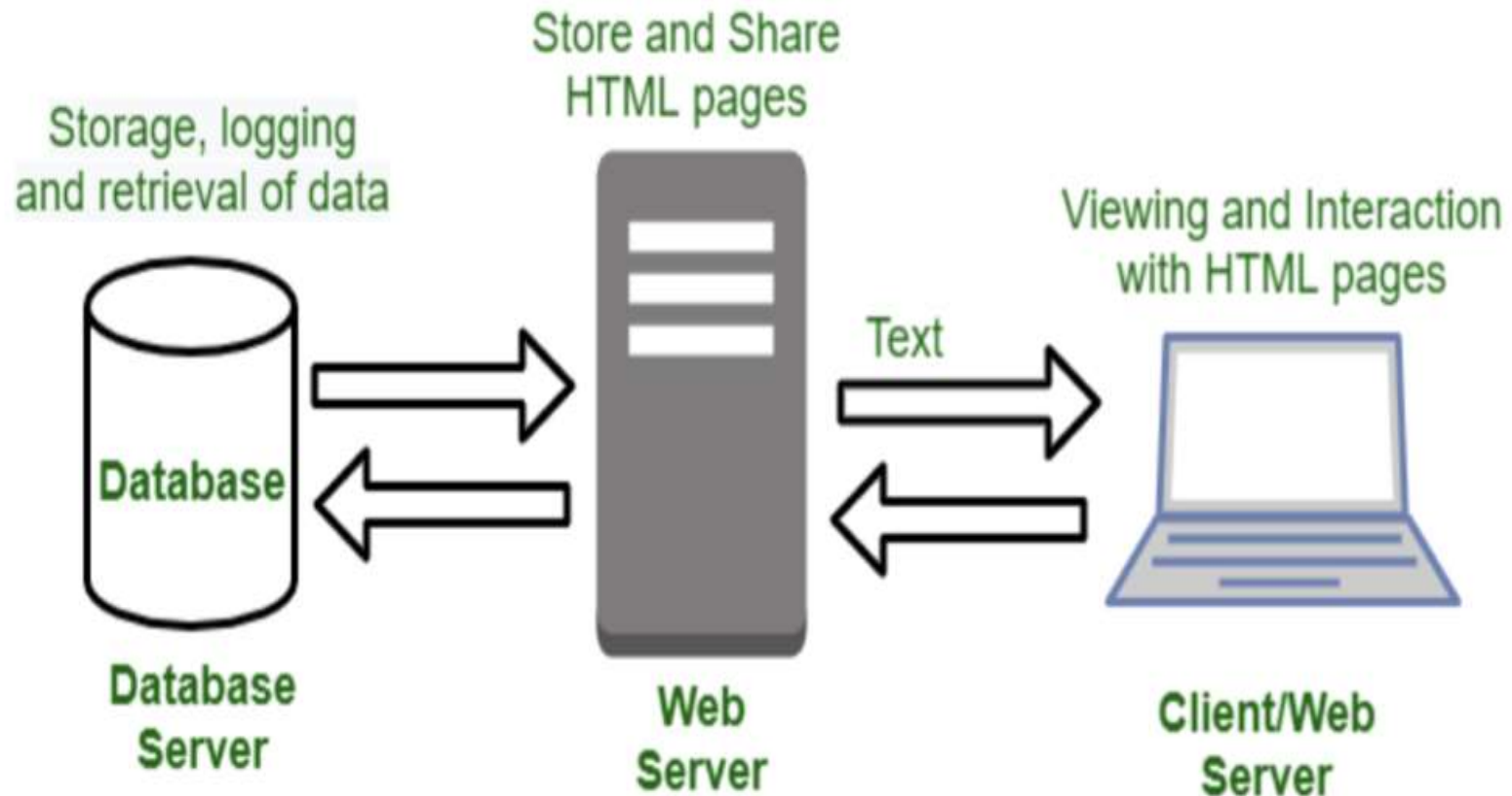
- In Static Websites, Web pages are returned by the server which are prebuilt source code files built using simple languages such as HTML, CSS, or JavaScript.
- There is no processing of content on the server (according to the user) in Static Websites.
- Web pages are returned by the server with no change therefore, static Websites are fast.
- There is no interaction with databases. Also, they are less costly as the host does not need to support server-side processing with different languages.
- Static does not mean that it will not respond to user actions, These Websites are called static because these cannot be manipulated on the server or interact with databases

Architecture of Static Website



- **Dynamic Website:** In Dynamic Websites, Web pages are returned by the server which are processed during runtime means they are not prebuilt web pages but they are built during runtime according to the user's demand with the help of server-side scripting languages such as PHP, Node.js, ASP.NET and many more supported by the server. So, they are slower than static websites but updates and interaction with databases are possible.
- Dynamic Websites are used over Static Websites as updates can be done very easily as compared to static websites (Where altering in every page is required) but in Dynamic Websites, it is possible to do a common change once and it will reflect in all the web pages.

Architecture of Dynamic Website



Difference Between Static and Dynamic Websites:

Static Website	Dynamic Website
Content of Web pages can not be change at runtime.	Content of Web pages can be changed.
No interaction with database possible.	Interaction with database is possible
It is faster to load as compared to dynamic website.	It is slower than static website.
Cheaper Development costs.	More Development costs.
No feature of Content Management.	Feature of Content Management System.
HTML, CSS, Javascript is used for developing the website.	Server side languages such as PHP, Node.js are used.
Same content is delivered everytime the page is loaded.	Content may change everytime the page is loaded.

Web Application Lifecycle



Cont..

Stage	Explanation
Planning	Defines the project objectives, requirements, timeline, and resources before development begins.
Design	Creates the user interface (UI), user experience (UX), and overall architecture of the web application.
Development	Implements the application by writing the front-end and back-end code.
Testing	Verifies that the application is free from bugs and meets all functional and performance requirements.
Deployment	Launches the completed web application to the live server for end users.
Maintenance	Continuously updates, fixes bugs, and improves the application after deployment.
Feedback	Collects user suggestions and reviews to identify areas for future improvements.
Monitoring	Continuously tracks the application's performance, security, and availability to ensure smooth operation.

What is a Web Browser?

- A Web Browser is a software application used to access, retrieve, and display web pages on the Internet. It communicates with web servers using HTTP/HTTPS protocols and renders HTML, CSS, and JavaScript into interactive web pages.

Popular Web Browsers

-  Google Chrome
-  Mozilla Firefox
-  Microsoft Edge
-  Safari
-  Opera

Browser Developer Tools (DevTools) are built-in utilities that help developers inspect, debug, and optimize websites.

Features

-  Inspect HTML elements
-  Modify CSS styles in real time
-  Debug JavaScript
-  Monitor network requests
-  Analyze website performance
-  Test responsive web design

What is HTTP?

- HTTP (HyperText Transfer Protocol) is a communication protocol used to transfer web pages and other resources between a web browser (client) and a web server.
- It is the foundation of communication on the World Wide Web (WWW).
- HTTP works on a request-response model.
- It is stateless, meaning every request is independent.

`http://www.example.com`

What is HTTPS?

HTTPS (HyperText Transfer Protocol Secure) is the secure version of HTTP.

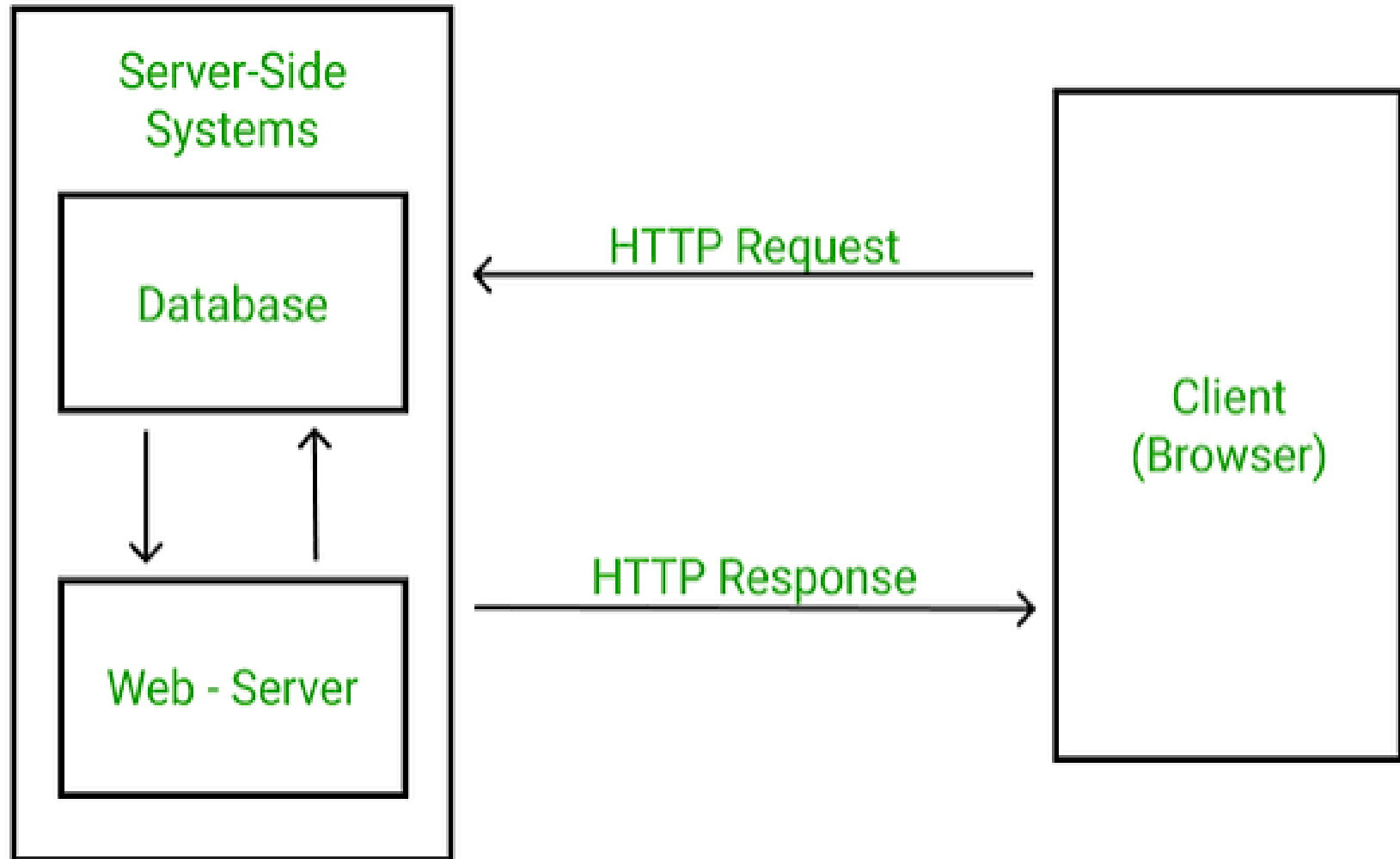
- It encrypts the data exchanged between the browser and the web server using SSL/TLS.
- It protects users from hackers, data theft, and tampering.
- Used for online banking, shopping, email, login pages, etc.
 - <https://www.example.com>

HTTP Methods

HTTP METHODS:

Method	Action	Example Use Case
GET	Retrieve data	View users list
POST	Create new resource	Register a new user
PUT	Update/replace data	Update full user profile
PATCH	Partial update	Update only email of a user
DELETE	Remove data	Delete a user account
HEAD	Metadata only	Check if file exists
OPTIONS	Show allowed methods	Preflight check (CORS) Cross Origin Resource Sharing

HTTP Request–Response Model



Difference between HTTP and HTTPS

Feature	HTTP	HTTPS
Full Form	HyperText Transfer Protocol	HyperText Transfer Protocol Secure
Security	Not Secure	Secure
Encryption	No encryption	Uses SSL/TLS encryption
Port Number	80	443
URL	http://	https://
Data Protection	Data can be intercepted	Data is encrypted
Authentication	No	Yes (SSL Certificate)
Speed	Slightly faster	Slightly slower due to encryption (difference is minimal today)
Used For	Normal websites	Banking, E-commerce, Login pages, Online payments

HTTP response status codes

HTTP response status codes indicate whether a specific HTTP request has been successfully completed. These are three-digit numbers sent by the server in response to a client's request.

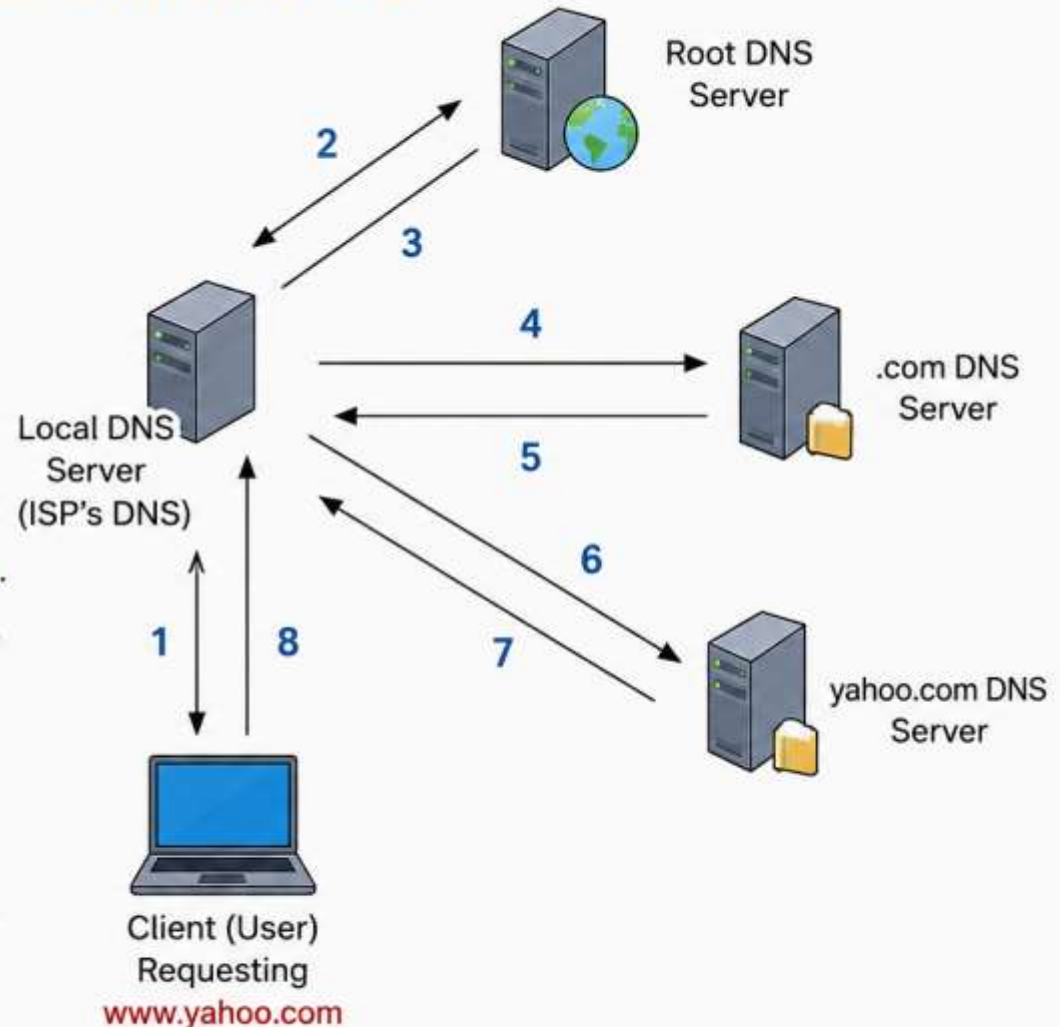
Responses are grouped in five classes:

Status Code	Class	Meaning
100 – 199	Informational Responses	The request has been received and processing can continue.
200 – 299	Successful Responses	The request was successfully received, understood, and processed.
300 – 399	Redirection Messages	Further action is required, usually to access the requested resource at another location.
400 – 499	Client Error Responses	There is an error in the request made by the client.
500 – 599	Server Error Responses	The server failed to fulfill a valid request.

- DNS (Domain Name System) is a system that translates human-readable domain names, like `www.example.com`, into machine-readable IP addresses, such as `192.168.1.1`, which computers use to identify and communicate with each other on the internet. Without DNS, we'd have to memorize IP addresses to access websites, making it difficult and inefficient for the average user.
- Another way, Domain name is a way to identify and locate computers connected to internet.
- No two organizations can have same domain name.
- A domain name always consists of two or more components separated by periods called dots (.).
- EXAMPLE: `www.yahoo.co.in`, `www.facebook.com` etc.
- "DOMAIN NAME SYSTEM is a collection of the databases that contain information about domain names and their corresponding IP address."

DNS EXAMPLE

- 1 User types **www.yahoo.com** in the browser. The request is sent to the local DNS server (at ISP's end).
- 2 The local DNS server does not have the IP address, so it forwards the request to the root DNS server.
- 3 Root DNS server replies: "I do not know the IP address of **www.yahoo.com**, but the **.com** DNS server knows."
- 4 The local DNS server then asks the **.com** DNS server for the IP address of **yahoo.com**.
- 5 The **.com** DNS server replies: "I do not know the IP address of **www.yahoo.com**, but the **yahoo.com** DNS server knows."
- 6 The local DNS server then asks the **yahoo.com** DNS server for the IP address.
- 7 The **yahoo.com** DNS server replies with the IP address of **www.yahoo.com**.
- 8 The local DNS server sends this IP address back to the client (user) that made the request.



Top-Level Domains (TLDs)

A Top-Level Domain (TLD) is the last part of a website address (domain name) that appears after the final dot (.). It indicates the type or purpose of the website or organization.

Each domain name has a corresponding IP address

When the user types the domain name in the address bar, the corresponding IP address is supplied. Such a translation is possible with the help of system called DNS (DOMAIN NAME SYSTEM)

LD	Full Form / Used For	Example
.com	Commercial organizations (Businesses)	www.amazon.com, www.yahoo.com
.net	Network organizations, Internet service providers	www.speedtest.net
.gov	Government organizations	www.india.gov.in (India), www.nj.gov (USA)
.edu	Educational institutions (mainly US universities)	www.harvard.edu
.org	Non-profit organizations	www.wikipedia.org
.mil	Military organizations (mainly US military)	www.army.mil
.int	International organizations	www.itu.int (International Telecommunication Union)

Geographical/Country Domains (ccTLDs)

- Geographical or Country Domains are two-letter domain extensions that represent a specific country or region.
- These domain codes are standardized by the International Organization for Standardization (ISO) under the ISO 3166-1 alpha-2 standard.
- They help identify the country to which a website belongs.

Domain	Country
• .in	India IN
• .jp	Japan JP
• .us	United States US
• .fr	France FR
• .it	Italy IT
• .cn	China CN
• .au	Australia AU

Website Structure

In web development, **structure** refers to the way a website's files, folders, pages, and resources are organized.

A well-structured website makes it easier to:

- Manage files and resources.
- Navigate between pages.
- Maintain and update the website.
- Deploy the website to a web server.
- Improve website performance and security.

```
mywebsite/  
|  
├─ index.html  
├─ about.html  
├─ contact.html  
|  
├─ css/  
|   └─ style.css  
|  
├─ js/  
|   └─ script.js  
|  
└─ images/  
    ├── logo.png  
    └─ banner.jpg
```

Important Files and Folders

File/Folder	Purpose
index.html	Main/home page
about.html	About page
contact.html	Contact page
css/	Contains CSS files
js/	Contains JavaScript files
images/	Contains images and other media

The **index.html** file is commonly used as the default page of a website.

URL



- URL stands for Uniform Resource Locator.
- A URL is the address of a resource on the Internet.

Example:



- **http** is the **scheme**
hyper text transfer protocol
- **two slashes** (`//`) separate the scheme from the machine/domain name
- **www.w3schools.com** is the **machine/domain name**
- **single slash** (`/`) separates the name from the path
- Finally **html/html_forms.asp** is the **path**.

Cont..



 The picture can't be displayed.

1. Protocol – Specifies how the browser communicates with the server.

- http
- https

2. Domain name – Identifies the website.

- example.com

3. Path – Identifies a specific resource.

- /products/index.html

4. Query parameters – Pass additional information to the server.

- ?id=101

Why Do We Need URLs?

URLs allow users and applications to:

1. Access websites.
2. Navigate between webpages.
3. Download files.
4. Load images and other resources.
5. Submit data to web applications.
6. Access APIs.
7. Share specific webpages with others.

Example

```
<a href="https://www.example.com">  
    Visit Example  
</a>
```

When the user clicks the link, the browser uses the URL to request the resource.

Absolute URL



An **absolute URL** contains the complete address of a resource, including the protocol and domain name.

Example

```
<a href="https://www.example.com/about.html">About Us</a>
```

Here:

`https://www.example.com/about.html` is an absolute URL.

Characteristics

- Contains the complete web address.
- Can point to a resource on another website.
- Does not depend on the location of the current webpage.

Example

```

```

When to Use Absolute URLs

Absolute URLs are commonly used when:

- Linking to another website.
- Referencing resources hosted on another domain.
- Sharing a complete web address.

Relative URL

A **relative URL** specifies the location of a resource relative to the current webpage.

Example:

```
<a href="about.html">About Us</a>
```

If the current page is:

```
https://www.example.com/index.html
```

the browser can resolve:

```
about.html
```

to:

```
https://www.example.com/about.html
```

Examples of Relative URLs

Same folder

```
<a href="about.html">About</a>
```

Subfolder

```
<a href="images/logo.png">Logo</a>
```

Parent folder

```
<a href="../index.html">Home</a>
```

.. means **move one directory up**.

Absolute URL Vs Relative URL

Absolute URL

- This type of URL contains both the domain name and directory/page path. An absolute URL gives complete location information. It begins with a protocol like "http://" and continues, including every detail.

Relative URL

- This type of URL contains the path excluding the domain name. Relative means "in relation to", and a relative URL tells a URL location on terms of the current location. Relative path is used for reference to a given link of a file that exist within the same domain.

Absolute URL	Relative URL
Contains full address	Contains only the path
Independent of current page	Depends on current page location
Used for external resources	Commonly used for internal resources
Example: https://example.com/page.html	Example: page.html

Query Parameters

Query parameters are pieces of information added to the end of a URL to send data to a web server.

They are written after a ? in the URL.

- <https://example.com/search?q=computer>
- Here:
 - `https://example.com/search` → URL
 - `?` → Starts the query parameters
 - `q` → Parameter name
 - `computer` → Parameter value
- So, `q=computer` means the search term is computer.

Multiple Query Parameters

Multiple parameters are separated using &.

- <https://example.com/products?category=mobile&brand=apple>
- Here:
- Parameter Value
- category mobile
- brand apple
- The server receives both pieces of information.

Web hosting is a service that provides storage and server resources for website files so that users can access the website through the Internet.

A website normally contains:

HTML + CSS + JavaScript + Images



Web Hosting Server



Internet



Web Browser

Simple Example

Think of a website as a house:

- **Website files** → Things inside the house
- **Hosting server** → The house
- **Domain name** → The address of the house

How Web Hosting Works

- A developer creates website files.
- The files are uploaded to a hosting server.
- The domain points users toward the hosting service.
- A user enters the domain in a browser.
- The browser sends a request to the server.
- The server returns the requested files.
- The browser displays the webpage.

Example

A website may have:

Domain: www.example.com

Hosting Server: Server storing website files

The domain helps users **find** the website, while hosting provides the place where the website **lives**.

Types of Web Hosting

There are several types of web hosting.

1. Shared Hosting

Multiple websites use the resources of the same server.

Server



Advantages

Low cost.

Easy to manage.

Suitable for small websites.

Disadvantages

Resources are shared.

Performance may be affected by other websites.

Limited server control.

Suitable for

Personal websites.

Small business websites.

Basic blogs.

2. VPS Hosting

VPS (Virtual Private Server) divides a physical server into multiple virtual servers.

Physical Server

- ┌— VPS 1 → Website A
- ┌— VPS 2 → Website B
- └— VPS 3 → Website C

Advantages

- More control
- Better performance than shared hosting
- Suitable for growing websites

Disadvantages

- More expensive than shared hosting.
- Requires more technical knowledge.

3. Dedicated Hosting

In **dedicated hosting**, an entire physical server is assigned to one customer.

Physical Server



One Website

Advantages

- High performance.
- Full server control.
- High level of customization.

Disadvantages

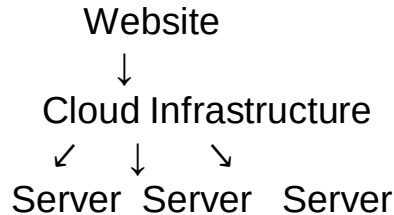
- Expensive.
- Requires server administration knowledge.

Suitable for

- Large websites.
- High-traffic applications.
- Organizations requiring dedicated resources.

4. Cloud hosting

Cloud hosting uses a network of interconnected servers to host websites and applications.



Advantages

- Scalable.
- High availability.
- Resources can be increased when required.

Disadvantages

- Pricing can be complex.
- Requires some technical knowledge for advanced configurations.

5. Managed Hosting

In **managed hosting**, the hosting provider handles many technical tasks such as:

- Server maintenance.
- Updates.
- Security configuration.
- Backups.
- Monitoring.

It is useful when the website owner does not want to manage server administration.

Domain vs Hosting

This is one of the most important concepts.

Domain

Domain = Address

Example:

`www.example.com`

Hosting

Hosting = Place where website files are stored

Example:

HTML

CSS

JavaScript

Images

Simple Analogy

Imagine a university campus:

Domain → Building address

Hosting → Building

Website files → Contents inside the building

So:

Domain tells users where to go, while hosting provides the place where the website is stored.

Domain and Hosting Integration

After purchasing a domain and hosting, they need to be connected.

Basic process:

Domain



DNS



Hosting Server



Website Files

Example

Suppose:

Domain:

www.mycollege.com

Website files are stored on a hosting server.

The domain's DNS settings are configured so that:

www.mycollege.com



Hosting Server



Website

Now, when a user enters:

<https://www.mycollege.com>

the website can be displayed.

Complete Concept

We can remember the complete process as:

Create Website



HTML + CSS + JS + Images



Upload Files to Hosting



Register Domain



Connect Domain with Hosting



DNS Configuration



User Enters Domain



Website Opens